ZenoDEX Whitepaper: Full-System Architecture, Assurance Boundaries, and Proof Agenda

Dana Edwards

April 2026

Abstract

ZenoDEX is intended to be a proof-carrying exchange architecture rather than a collection of ad hoc trading or settlement scripts. The complete system spans theorem-bearing spot kernels, certificate-carrying settlement and price-provenance planes, kernel-composition contracts, bounded stablecoin and perpetual-risk slices, narrow runtime ingress boundaries, and advisory automation that remains structurally non-authoritative. This whitepaper states the intended full system, the current evidence frontier, the narrower current RC1 public claim surface, and the remaining proof agenda required to close the gap between them. The newest proof layer adds a mechanism-generic settlement theorem and a mistake-proofing theorem: any mechanism that supplies exact generator coverage, a certified winner, feasibility, and trace realization inherits canonical safe settlement, while dangerous user actions may enter the submit path only through an explicit interlock.

Contributions. This whitepaper contributes:

1. a full-system decomposition of ZenoDEX into execution, guard, certificate, risk, runtime, and assurance planes;
2. an explicit evidence discipline for separating proved, contract-backed, implemented, tested-discovery, and hypothetical claims;
3. the global settlement-algebra theorem schema tying runtime candidate generation, winner certificates, and trace realization to canonical safe settlement;
4. the Poka-yoke safety theorem tying dangerous status classes to explicit user interlocks;
5. a guarantee-and-assumption matrix for the main protocol surfaces;
6. an artifact and algorithm map that ties paper claims back to concrete repo-local files.

1 Claim discipline and system shape

The central architecture claim is

$$\begin{aligned}\mathsf{ZenoDEX} := \mathsf{Exec} \wedge \mathsf{Guards} \wedge \mathsf{Cert} \wedge \mathsf{Control} \wedge \mathsf{Oracle} \\ \wedge\, \mathsf{zUSD} \wedge \mathsf{Perps} \wedge \mathsf{Runtime} \wedge \mathsf{Assurance}.\end{aligned}$$

Standard reading: the full ZenoDEX is the conjunction of execution kernels, guard systems, certificate systems, control planes, oracle and monetary layers, runtime adapters, and assurance artifacts. Practical consequence: the system should be judged as an architecture with explicit proof and contract boundaries, not as one isolated spot-exchange algorithm.

The current full-stack shaping formula recorded in the repo is

$$\begin{aligned}\mathsf{Shape}(\mathsf{ZenoDEX}) := \mathsf{TauGuards} \wedge \mathsf{TLAShadows} \wedge \mathsf{LeanProofObjects} \\ \wedge\, \mathsf{RuntimeIntegerArithmetic} \wedge \mathsf{RoutingObjectivesAndCertificates} \\ \wedge\, \mathsf{StablecoinSolvencyAndOracleSafety}.\end{aligned}$$

The evidence discipline is

$$\texttt{proved} > \texttt{contract} > \texttt{implemented} > \texttt{tested\_discovery} > \texttt{hypothesis}.$$

Standard reading: each claim may only be stated at the strongest replayable evidence class actually present. Practical consequence: the whitepaper can describe the built-out target system, but it cannot collapse implemented or experimental slices into formal proofs.

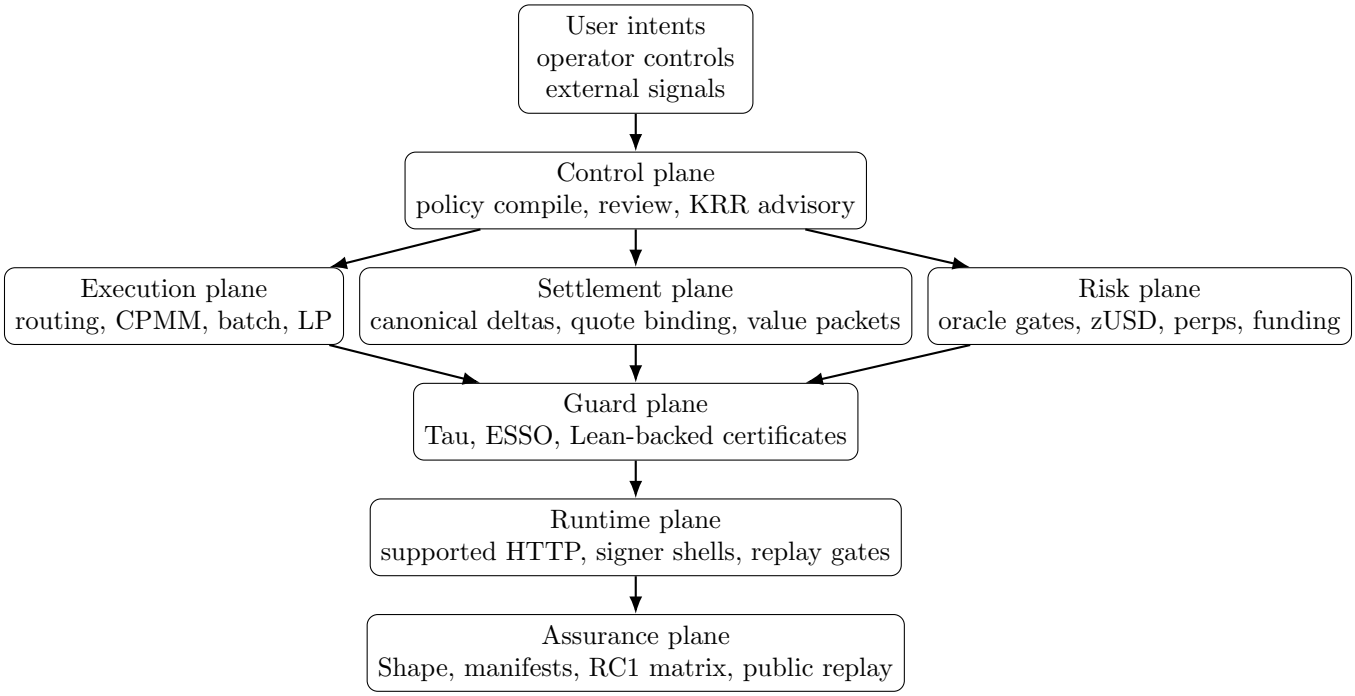

Figure 1: The intended full ZenoDEX stack. Advisory logic may shape ranking and explanations, but execution authority only travels through fail-closed guards and narrow runtime adapters.

2 World model and cross-slice invariants

The promoted ShapeForge world model is a typed object

$$\mathsf{World} := \langle \mathsf{Slices}, \mathsf{CrossSliceInvariants}, \mathsf{ScenarioTransforms} \rangle.$$

Important cross-slice laws already recorded in that world model include:

$$\mathsf{CanonicalWinnerClaim} \to \mathsf{TotalKeyOnEveryAdmittedCandidateSet},$$

$$\mathsf{RoutingFeasible} \vee \mathsf{BatchFeasible} \to \mathsf{CPMMSafetyPreconditions},$$

$$\mathsf{SettlementCertificateMeaningful} \to \mathsf{CanonicalDeltas} \wedge \mathsf{ReplayBoundWitnesses}.$$

Standard reading: the current world model already ties winner claims to total keys, routing and batch feasibility to local AMM safety, and settlement meaning to canonical deltas and replay-bound witnesses. Practical consequence: the repo already has a system-level semantics discipline, not just isolated code modules.

3 Execution plane and theorem-bearing spot core

The strongest current theorem stock lives in the bounded spot core.

3.1 Canonical winner semantics

The central selection law is

$$C \neq \varnothing \to \exists!\, w \in C \text{ such that } \forall c \in C,\ \mathsf{Key}(w) \leq \mathsf{Key}(c).$$

Standard reading: every admitted nonempty candidate family with a total key has a unique canonical winner. Practical consequence: canonicality is a semantic property, not merely a stable sort artifact.

Claim 1 (Current strongest spot theorem posture)**.** *For the promoted exact-in routing and batch-clearing slices, the repository already contains Lean-backed winner theorems strong enough to justify canonical-winner claims on the emitted bounded candidate families.*

Proof idea. The current Lean files prove unique canonical minima for finite batch candidate sets and prove the exact-in ranked-witness shell that transports route-key order into certificate structure. This does not yet prove universal optimality over every conceivable route family; it proves canonicality over the actual bounded emitted candidate family.

3.2 CPMM safety and fee conservation

The local AMM safety envelope is shaped by

$$\mathsf{amount\_out} < \mathsf{reserve\_out}, \qquad K_{after} \geq K_{before} \text{ (zero-fee integer model)},$$

and the fee-side conservation posture is

$$\mathsf{fee\_conserved} \wedge \mathsf{dust\_carry\_bounded}.$$

Standard reading: spot safety depends on reserve-bounded swaps, local invariant structure, and explicit dust state when exact fee conservation matters. Practical consequence: higher-level routing and batch correctness should be stated only over states that already satisfy these local preconditions.

4 Certificate, settlement, and composition planes

ZenoDEX is intended to move from state transitions to witness-carrying transitions. The settlement composition law is

$$\mathsf{SettlementOK} \to \mathsf{CanonicalDeltaTable} \wedge \mathsf{QuoteOrKernelWitnessBound} \wedge \mathsf{ReplayVerifiable}.$$

Standard reading: a settlement object is meaningful only if its deltas are canonical, its fills are witness-bound, and the packet can be replay-verified. Practical consequence: settlement should be read as a certificate plane rather than a post hoc report.

4.1 Settlement algebra and global mechanism theorem

The underlying additive settlement object remains the same small algebra:

$$\mathsf{Settlement} := \mathbb{Z} \times \mathbb{Z}, \qquad (\Delta x_1, \Delta y_1) \circ_s (\Delta x_2, \Delta y_2) := (\Delta x_1 + \Delta x_2, \Delta y_1 + \Delta y_2),$$

with scalar net-flow map

$$\Delta(\Delta x, \Delta y) := \Delta x + \Delta y.$$

Standard reading: a two-asset settlement is a signed reserve delta pair, settlements compose by componentwise addition, and Δ is the scalar net-flow homomorphism. Practical consequence: this algebra is useful for certificate composition and replay, but scalar net-flow is not an economic value invariant because it adds unlike token units.

The newer result is not a change to that algebra. It is a global theorem schema for mechanisms that want to reuse the canonical settlement proof. A mechanism M exposes a finite feasible domain, an emitted runtime key list, a selected candidate, and the trace actually executed. The obligation bundle is

$$\begin{aligned}&\mathsf{GeneratorExact}(M) \wedge \mathsf{WinnerCertified}(M) \wedge \mathsf{WinnerFeasible}(M)\\ &\wedge \mathsf{TraceRealizesWinner}(M) \to \mathsf{CanonicalSafeSettlement}(M).\end{aligned}$$

Standard reading: if the emitted list is exactly the feasible key image, the selected candidate is certified as the winner of that list, the selected key is feasible, and the runtime trace realizes that selected candidate, then the trace executes the canonical safe settlement for M. Practical consequence: CPMM batches, route selection, orderbook settlement, perp settlement, and future DEX mechanisms do not need to reprove the canonical settlement theorem from scratch; they must prove that they instantiate this interface.

The conclusion is not a vague safety label. In the current Lean theorem it expands to:

$$S_M := \mathsf{feasibleKeys}(M), \quad A_M := \pi_1(\mathsf{batchAB}(M.\mathsf{trace})), \quad B_M := \pi_2(\mathsf{batchAB}(M.\mathsf{trace})).$$

$$\begin{aligned}\mathsf{CanonicalSafeSettlement}(M) \iff\ & \mathsf{batchToSettlement}(M.\mathsf{trace}) = \mathsf{batchToSettlement}(M.\mathsf{selected.batch})\\ &\wedge\ M.\mathsf{selected.key} \in S_M\\ &\wedge\ \forall x \in S_M,\ A_M \geq \mathsf{vol}(x)\\ &\wedge\ \forall x \in S_M,\ A_M = \mathsf{vol}(x) \to B_M \geq \mathsf{sur}(x)\\ &\wedge\ \forall x \in S_M,\ A_M = \mathsf{vol}(x) \to B_M = \mathsf{sur}(x)\\ &\qquad\qquad \to M.\mathsf{selected.order} \leq \mathsf{ord}(x).\end{aligned}$$

Standard reading: the executed trace has the selected settlement, the selected key is feasible, and the trace is lexicographically maximal by volume, then surplus, with minimum order among exact ties over the feasible key image.

The proof-carrying bridge has the concrete certificate shape

$$\mathsf{ListCertificateOK}(L, w) \leftrightarrow w \in L \wedge \forall x \in L,\ w \leq x,$$

and the generator audit shape

$$L.\mathsf{toFinset} = \mathsf{image}(\mathsf{keyOf}, D).$$

Standard reading: the winner is a member of the emitted candidate-key list and is minimal under the explicit key; after deduplication, the emitted list equals the image of the feasible domain. Practical consequence: the runtime search is not trusted as an optimizer. It is trusted only to emit replayable evidence that the Lean theorem can check.

The composition law across kernels is the mirror-sync discipline:

$\mathsf{SharedFactInA} = \mathsf{MirrorOfFactInB}$ must hold inductively under each reserve-affecting or stake-affecting step.

Claim 2 (Composition invariant)**.** *Composable kernel invariants remain inductive only when shared reserve and stake mirrors are synchronized after every transition that can change the source fact.*

Proof idea. Without synchronized mirrors, a downstream guard reasons over stale pre-state. With explicit mirror wiring and system invariants, inductiveness is restored.

The value and oracle packet side is similarly explicit:

$$\mathsf{AssetConserved} \wedge \mathsf{ExplicitNonnegativePrices} \rightarrow \mathsf{NetSpotValue} = 0,$$

$$\mathsf{PricePacketAdmissible} \leftrightarrow \mathsf{UniqueAssetSet} \wedge \mathsf{PositivePrices} \wedge \mathsf{FreshEnough} \wedge \mathsf{SyncGateGreen}.$$

Standard reading: value-neutrality and price provenance are explicit contract surfaces with explicit assumptions. Practical consequence: the whitepaper can state these guarantees precisely instead of vaguely claiming "oracle safety."

5 Risk planes: zUSD, perps, and advisory automation

The full architecture includes monetary and derivatives layers even though their proof maturity differs from the spot core.

The intended zUSD invariant is

$$\mathsf{zUSDAdmissible} \rightarrow \mathsf{DebtConservation} \wedge \mathsf{CollateralConservation},$$

with a stale-oracle barrier of the form

$$\mathsf{price\_pending} \neq \mathsf{price} \rightarrow \mathsf{RiskyTransitionsBlocked}.$$

For perps, the intended posture is

$$\mathsf{PerpSettlementAdmissible} \rightarrow \mathsf{FreshOracleWindow} \wedge \mathsf{BoundedMoveGate} \wedge \mathsf{FundingWitnessLaneValid}.$$

Standard reading: the monetary and derivatives slices are bounded by explicit oracle freshness and settlement-witness conditions. Practical consequence: these layers belong in the whole-system architecture, but they are not yet as mature as the theorem-bearing spot core.

Automation is present but bounded by a strict trust law:

$$\mathsf{KRR} \rightarrow \mathsf{AdvisoryOnly}, \qquad \neg(\mathsf{KRR} \rightarrow \mathsf{ExecutionAuthority}).$$

Standard reading: KRR may rank checks and summarize trust posture, but it never becomes a value-moving authority. Practical consequence: the system can include advisory intelligence without collapsing the core correctness boundary.

5.1 Poka-yoke safety contract

The mistake-proofing layer is formalized as a fail-closed submit predicate. For a risk status r and interlock state i,

$$\mathsf{submitAllowed}(r, i) = \mathsf{true} \iff (\mathsf{dangerous}(r) = \mathsf{false}) \vee (\mathsf{interlockSatisfied}(i) = \mathsf{true}),$$

where

$$\begin{aligned}\mathsf{interlockSatisfied}(i) = \mathsf{true} \iff \mathsf{interlockPresent}(i) = \mathsf{true} \\ \wedge \big(\mathsf{typedConfirmAccepted}(i) = \mathsf{true} \\ \vee\ \mathsf{advancedOverrideLogged}(i) = \mathsf{true}\big).\end{aligned}$$

The Lean theorem is

$$\mathsf{submitAllowed}(r, i) = \mathsf{true} \to \mathsf{Shielded}(r, i).$$

The shielded predicate expands to

$$\begin{aligned}\mathsf{Shielded}(r, i) \iff \mathsf{dangerous}(r) = \mathsf{false} \\ \vee\ \mathsf{interlockPresent}(i) = \mathsf{true} \wedge \big(\mathsf{typedConfirmAccepted}(i) = \mathsf{true} \\ \vee\ \mathsf{advancedOverrideLogged}(i) = \mathsf{true}\big).\end{aligned}$$

Standard reading: if a submit is allowed, then either the action is not dangerous or an explicit interlock exists and was satisfied by typed confirmation or a logged advanced override. Practical consequence: the user-interface and audit tools can add new failure modes without weakening the safety contract, provided each dangerous mode has a visible signal and a submit interlock.

The runtime audit report mirrors the theorem with the obligation

$$\mathsf{signal\_present} \wedge \mathsf{interlock\_present}$$

for every catalogued dangerous failure mode. The current audit contract is replayable through `tools/pokayoke/pokayoke_audit.py`; it fails closed if a catalogued mode is signal-only, interlock-only, or uncovered.

6 Guarantees and explicit assumptions

The right security statement for ZenoDEX is a matrix of guarantees under assumptions.

7 Promotion ledger and proof obligations

A full-system paper is only useful if it states how each major slice can be promoted rather than merely asserting that promotion should happen later. The promotion discipline is

$$\mathsf{PromotionReady}(s) := \mathsf{HonestCurrentClaim}(s) \wedge \mathsf{ExplicitGap}(s) \wedge \mathsf{ReplayablePromotionArtifact}(s).$$

Standard reading: a slice is promotion-ready only when the current claim is stated honestly, the missing proof obligation is explicit, and there is a concrete artifact lane that could discharge it. Practical consequence: the whitepaper should separate architecture from wishful proof narratives.

8 Public boundary versus full-system scope

The public-release boundary is intentionally narrower than the whitepaper scope:

$$\mathsf{RC1ClaimOK} := \mathsf{CleanTree} \wedge \mathsf{ScopeFrozen} \wedge \mathsf{ReplayGreen} \wedge \mathsf{ExclusionsHonest}.$$

This yields the three-level relation

$$\mathsf{FullTargetArchitecture} \supset \mathsf{CurrentEvidenceFrontier} \supset \mathsf{CurrentRC1Claim}.$$

Standard reading: the complete intended ZenoDEX is broader than the claims currently backed by the strongest evidence, and those claims are broader than the narrow public RC1 release claim. Practical consequence: the whitepaper can describe the full target architecture while still preserving an honest current release boundary.

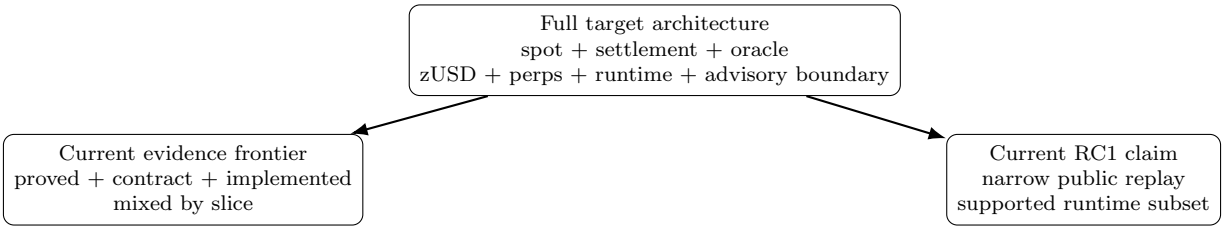

Figure 2: Assurance boundary picture. The target architecture is broader than the current evidence frontier, and the current RC1 public claim is narrower still.

9 Artifact map

Table 3 compresses the main artifact families behind the whitepaper claims.

10 Reference algorithm skeletons

The whitepaper should expose the control flow of the main bounded procedures as well as their supporting artifacts.

A. Canonical bounded route selection.

$$\mathsf{Winner}(C) := \arg\min_{c \in C} \mathsf{Key}(c).$$

Operational skeleton:
1. emit a bounded candidate family C from the current admissible route domain;
2. compute the explicit runtime key for each candidate;
3. select $w = \mathsf{Winner}(C)$;
4. build the certificate / projection / guarded quote packet around w;
5. expose the quote as canonical only when the guarded packet verifies.

Primary artifacts:
- `lean-mathlib/Proofs/ZenoDEXExactInTrueKeyWinner.lean`
- `src/integration/exact_in_route_certificate.py`
- `src/integration/tau_witness.py`

B. Settlement end-to-end certificate build.

$$\mathsf{packet\_ok} \leftrightarrow \mathsf{strong\_certificate\_ok} \wedge \mathsf{feature\_extension\_ok} \wedge \mathsf{value\_lane\_ok}.$$

Operational skeleton:
1. build the strong settlement certificate on the canonical delta table;
2. attach the feature-extension packet and chosen value lane;
3. attach provenance or signed-attestation inputs as required;
4. serialize the packet;
5. verify the bundle-level shell by re-checking each component gate.

Primary artifacts:
- `src/integration/settlement_end_to_end_certificate_packet.py`
- `src/kernels/dex/settlement_end_to_end_certificate_packet_v1.yaml`
- `lean-mathlib/Proofs/ZenoDEXSettlementEndToEndCertificatePacket.lean`

C. Global mechanism settlement acceptance.

$$\mathsf{AcceptedByFamily}(F, M) \to \mathsf{CanonicalSafeSettlement}(M).$$

Operational skeleton:
1. define the mechanism's feasible domain D and key function keyOf;
2. emit the runtime candidate-key list L;
3. prove $L.\mathsf{toFinset} = \mathsf{image}(\mathsf{keyOf}, D)$;
4. prove the selected key is list-minimal and feasible;

5. prove the executed trace realizes the selected candidate;
6. apply the global settlement theorem.

Primary artifacts:

- `lean-mathlib/Proofs/SettlementCanonicalExecution.lean`
- `lean-mathlib/Proofs/SettlementMechanism.lean`

D. Poka-yoke submit gate.

$$\mathsf{dangerous}(r) = \mathsf{true} \wedge \mathsf{interlockSatisfied}(i) = \mathsf{false} \rightarrow \mathsf{submitAllowed}(r, i) = \mathsf{false}.$$

Operational skeleton:

1. classify the current action into a risk status;
2. render a user-visible signal for each dangerous status;
3. require a typed confirmation or logged advanced override before submit;
4. emit an audit report requiring both signal and interlock coverage for every catalogued mode.

Primary artifacts:

- `lean-mathlib/Proofs/PokayokeSafety.lean`
- `tools/pokayoke/pokayoke_audit.py`

E. Advisory autotrader live preparation.

$$\mathsf{LiveEmitAllowed} \rightarrow \mathsf{PolicyContractOK} \wedge \mathsf{ReceiptOK} \wedge \mathsf{SignerNonceOK} \wedge \mathsf{ShellGuardsOK},$$

with

$$\mathsf{KRR} \rightarrow \mathsf{AdvisoryOnly}.$$

Operational skeleton:

1. load and verify the signed policy bundle or artifact chain;
2. build the bounded observation packet from trusted and advisory signals;
3. compute KRR advice if available, otherwise degrade to a report with advisory error details;
4. enforce local guards, receipt checks, signer binding, nonce sequencing, and live-shell admission;
5. emit either a structured reject / skip report or a signed live-preparation object.

Primary artifacts:

- `tools/autotrader_policy_compile.py`
- `src/agents/krr_policy_advisor.py`
- `src/integration/autotrader_live.py`
- `tools/autotrader_live.py`

11 End-state proof agenda

The completed ZenoDEX should satisfy the stronger conjunction

$$\begin{aligned}\mathsf{FinishedZenoDEX} := \mathsf{SpotCanonicality} &\wedge \mathsf{SettlementConservation} \wedge \mathsf{OracleProvenanceSafety} \\ &\wedge \mathsf{zUSDSolvency} \wedge \mathsf{PerpRiskContainment} \wedge \mathsf{TypedRuntimeIngress} \\ &\wedge \mathsf{AdvisoryOnlyAutomationBoundary}.\end{aligned}$$

The highest-value remaining obligations are:

1. instantiate the global settlement mechanism theorem for each production mechanism family, rather than merely documenting that the interface exists;
2. complete broader route enumeration and AMM semantics bridges so canonical route claims depend less on runtime-backed candidate generation;
3. connect concrete Poka-yoke UI and API submit surfaces to the generic fail-closed submit theorem, keeping the audit catalog green as new risks are added;
4. raise the zUSD and perps planes from mixed evidence to a cleaner theorem/contract split with explicit public settlement semantics;
5. preserve the advisory-only automation boundary even as KRR and higher-level authoring grow.

12 Conclusion

The narrow spot proof note is not the whole ZenoDEX, and the current RC1 surface is not the whole current evidence frontier. The correct whole-system story is a proof-carrying architecture with theorem-bearing spot kernels at its center, fail-closed certificate and composition layers around them, bounded risk planes, narrow runtime ingress, and advisory automation that remains structurally unable to seize execution authority.

Primary repo-local anchors. This whitepaper is grounded in the promoted ShapeForge world model, the RC1 replay and verified-surface notes, the automation boundary note, the kernel-composition note, and the theorem-bearing Lean files and kernel specs named throughout this document.

References

[1] Promoted ShapeForge world model seed, `docs/zenodex/shapeforge_promoted/zenodex_world_model.seed.json`.

[2] ShapeForge reasoning note, `docs/zenodex/SHAPEFORGE_ZENODEX_REASONING.md`.

[3] RC1 verified surface matrix, `docs/RC1_VERIFIED_SURFACE_MATRIX.md`.

[4] Public assurance replay note, `docs/PUBLIC_ASSURANCE_REPLAY.md`.

[5] Kernel ABI and composition note, `docs/KERNEL_ABI_AND_COMPOSITION.md`.

[6] Settlement algebra paper package, `docs/papers/settlement-algebra-batch-cpmm/main.tex`.

[7] Global settlement mechanism theorem, `lean-mathlib/Proofs/SettlementMechanism.lean`.

[8] Poka-yoke safety theorem, `lean-mathlib/Proofs/PokayokeSafety.lean`.

[9] Autotrader KRR boundary note, `docs/AUTOTRADER_KRR.md`.

[10] Companion appendix, `docs/papers/zenodex-full-system/appendix.tex`.

Surface	Intended guarantee	Current strongest evidence	Assumptions / limits
Spot winner selection	unique canonical winner on emitted bounded candidates	proved	explicit total key, nonempty admitted candidate set, bounded emitted family
Spot CPMM / fee core	reserve-bounded swap and exact fee-accounting posture	proved	zero-fee monotonicity theorem is model-specific; exact conservation needs explicit dust state
Settlement / quote binding	replay-verifiable canonical settlement packet	proved bridge + contract	global theorem requires exact generator coverage, winner certificate, winner feasibility, and trace realization
Value and price packets	explicit value-neutrality and provenance admissibility contracts	contract + tests	declared price vector and freshness policy are trusted contract inputs
Kernel composition	inductive preservation of shared facts across modules	contract	requires explicit mirror-sync wiring, not eventual consistency
Perps / zUSD	bounded solvency and oracle safety posture	mixed	not all claims are public-release-grade today
Poka-yoke submit safety	dangerous submit paths require explicit interlocks	proved + audit contract	concrete surfaces must instantiate the fail-closed predicate and keep catalog coverage green
Autotrader / KRR	advisory automation with fail-closed shell	contract + implemented	experimental, out of RC1 authority, never execution-authoritative
Public RC1 release	narrow replayable public assurance surface	contract + replay evidence	clean tree, frozen scope, green replay lanes, exclusions remain excluded

Table 1: Whole-system guarantee matrix. The architecture is broad, but the current evidence class and assumption boundary vary by surface.

Surface	Honest current statement	Promotion obligation	Likely artifact lane
Exact-in routing	canonical winner over emitted bounded candidates	justify emitted-family sufficiency or prove broader completeness over the intended route domain	Lean selector theorem + bounded enumerator contract
Settlement packets	global canonical settlement theorem exists for mechanisms satisfying the obligation interface	instantiate exact generator coverage and trace-realization obligations for each production mechanism family	`SettlementMechanism.lean` + mechanism-specific Lean/ESSO/Tau certificates
Kernel composition	mirror-sync composition contract is explicit	prove inductiveness of chosen composed systems under all reserve- and stake-affecting transitions	ESSO/TLA+/Lean composition proofs
zUSD / perps	bounded solvency and oracle-safety posture exist but evidence is mixed	promote monetary and derivatives semantics into public theorem/contract surfaces with replay witnesses	Lean/ESSO kernels + replay manifolds
Poka-yoke	generic submit-safety theorem and audit contract exist	prove each concrete surface computes or refines the fail-closed predicate	Lean theorem + replayed audit contract + UI/API tests
Autotrader / KRR	advisory-only shell is explicit and fail-closed at runtime boundary	preserve non-authority while making trust ranking, calibration, and higher-level authoring more explicit	typed contracts + bounded advisory tests

Table 2: Per-slice promotion ledger. The goal is not to claim future proofs now, but to state the concrete artifact lane that would justify a stronger claim later.

Claim family	Evidence class	Primary artifacts
Canonical spot winners	proved	`lean-mathlib/Proofs/BatchAuctionCanonical.lean`; `ZenoDEXExactInTrueKeyWinner.lean`; `ZenoDEXExactInRouteRankProjection.lean`
CPMM and fee core	proved	`lean-mathlib/Proofs/CPMMInvariants.lean`; `FeeDustCarryConservation.lean`; `OppositeDirectionNoncommutativity.lean`
Settlement algebra and mechanism plane	proved theorem schema + contract adapters	`lean-mathlib/Proofs/SettlementAlgebra.lean`; `SettlementCanonicalExecution.lean`; `SettlementMechanism.lean`; `BatchCPMMUnification.lean`; `src/kernels/dex/settlement_end_to_end_certificate_packet_v1.yaml`
Poka-yoke submit safety	proved theorem + replayed audit contract	`lean-mathlib/Proofs/PokayokeSafety.lean`; `tools/pokayoke/pokayoke_audit.py`; `tests/tools/test_pokayoke_audit_tool.py`
Composition and release boundary	contract	`docs/KERNEL_ABI_AND_COMPOSITION.md`; `src/kernels/dex/zenodex_system_compose_v2.yaml`; `docs/RC1_VERIFIED_SURFACE_MATRIX.md`; `docs/PUBLIC_ASSURANCE_REPLAY.md`
Risk, monetary, and automation boundary	mixed	`docs/derivatives/PERP_EPOCH_SAFETY_V1.md`; `src/kernels/dex/perp_epoch_isolated_v2.yaml`; `docs/AUTOTRADER_KRR.md`; `src/agents/krr_policy_advisor.py`

Table 3: Condensed artifact map. The companion appendix expands these rows into a larger audit ledger.